

Universidad Nacional de Lomas de Zamora

Facultad de Ingeniería

Proyecto FINAL - Estacion de calidad por Perfilometria

Alumno: QUINTANA, Fernando Miguel 

In []:

```
1 # -----
2 # Importamos librerías
3 import numpy as np
4 import cv2
5 import matplotlib.pyplot as plt
6
7 # Cargamos los archivos de las matrices de posición X, Y y Z
8 mtx_xi = np.loadtxt("mtx_x_izq.txt") # Matriz pos. píxeles en X
9 mtx_yi = np.loadtxt("mtx_y_izq.txt") # Matriz pos. píxeles en Y
10 mtx_zi = np.loadtxt("mtx_z_izq.txt") # Matriz pos. píxeles en Z
11
12 # Creamos las variables auxiliares a utilizar en la ejecución del programa
13 scale_factor_x = 1 # Escalado en X
14 scale_factor_y = 2.3 # Escalado en Y
15 scale_factor_z = 2.3 # Escalado en Z
16
17
18 # Creamos los offset en X, Y y Z para mover el gráfico.
19 offset_x = 0
20 offset_y = -18
21 offset_z = 0
22
23 # Creamos la función para filtrado de "ROJO" mediante HSV
24 def HSV_FILTRO(image):
25     # Duplicamos la imagen
26     result = image.copy()
27     # Convertimos la imagen a espacio de color HSV
28     hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
29     # Definimos los límites inferiores y superiores del color rojo en HSV
30     lower_red = np.array([0, 50, 50])
31     upper_red = np.array([10, 255, 255])
32     # Creamos la máscara para el color rojo
33     mask1 = cv2.inRange(hsv_image, lower_red, upper_red)
34     # Definimos un rango adicional para el color rojo debido a su disposición en el espacio de color HSV
35     lower_red = np.array([170, 50, 50])
36     upper_red = np.array([180, 255, 255])
37     # Creamos otra máscara para el color rojo
38     mask2 = cv2.inRange(hsv_image, lower_red, upper_red)
39     # Combinamos ambas máscaras para obtener una única máscara para el rojo
40     mask = cv2.bitwise_or(mask1, mask2)
41     # Aplicamos la máscara a la imagen original para obtener solo los píxeles rojos
```

```
42     result = cv2.bitwise_and(result, result, mask=mask)
43     # Convertimos la imagen resultante a escala de grises
44     img_gris = cv2.cvtColor(result, cv2.COLOR_BGR2GRAY)
45     # Retornamos la imagen filtrada
46     return img_gris
47
48 # Creamos la función para calibración de imagen
49 def CALIBRAR(imagen, h, w):
50     # PROPIEDADES INTRÍNSECAS DE CÁMARA
51     # Matriz intrínseca de Cámara
52     M_int = np.array([[1.38106997e03, 0.00000000e00, 6.87317037e02],
53                       [0.00000000e00, 1.38209804e03, 3.82421364e02],
54                       [0.00000000e00, 0.00000000e00, 1.00000000e00]])
55
56     # Matriz de Distorsión
57     D = np.array([8.30863128e-02, 1.57413825e00, 4.52467136e-04, 5.90615097e-03, -7.07611874e00])
58
59     # Matriz Transformación de Cámara - Pixel
60     newcameramtx, roi = cv2.getOptimalNewCameraMatrix(M_int, D, (w, h), 1, (w, h), centerPrincipalPoint=False)
61
62     # Matriz Imagen calibrada
63     dst = cv2.undistort(imagen, M_int, D, None, newcameramtx)
64
65     # Se ajusta tamaño de imagen
66     x, y, w, h = roi
67     dst = dst[y : y + h, x : x + w]
68
69     # Retorna imagen calibrada
70     return dst
71
72 # Creamos la función para procesar la imagen
73 def PROCESAR(image):
74     # Inicializamos las variables a utilizar
75     perfil_puntos = [] # Lista de puntos del perfil de la pieza
76     pos_pix2 = [] # Lista de cada pixel a encontrar en las matrices de referencia
77
78     # Obtenemos el tamaño de la imagen recibida
79     h, w = image.shape[:2]
80
81     # Procesamos la imagen -> calibramos y filtramos "ROJO"
82     image = CALIBRAR(image, h, w)
83     image = HSV_FILTRO(image)
```

```
84
85     # Aplicamos un tresh para poner los pixeles en 1 o 0 según los valores límites fijados
86     image = cv2.threshold(image, 15, 255, cv2.THRESH_BINARY)[1]
87
88     # Mostramos imagen con umbral aplicado
89     cv2.imshow("Imagen", image)
90     cv2.waitKey(0)
91
92     # Recorremos todas las columnas de la imagen, para asegurarnos obtener una linea lo más fina posible a la curva
93     img_columnas = len(image[0]) # Obtenemos la cantidad de columnas que tiene la imagen
94
95     for col in range(0, img_columnas):
96         vpx = np.where(image[:, col] != 0) # Aquellos pixeles con "1" son los pixeles que queremos ubicar
97
98         # Si para dicha columna tenemos pixeles en "1", entonces lo guardamos en la lista
99         if len(vpx[0]) > 0:
100             # Ubicamos el pixel más centrado de la linea para ajustar lo más posible.
101             if len(vpx[0]) % 2 == 0:
102                 pos_pix2.append([vpx[0][int(len(vpx[0]) / 2)], col])
103             else:
104                 pos_pix2.append([vpx[0][int(round(len(vpx[0]) / 2, 0))], col])
105
106     # Para cada posición encontrada, se obtienen los valores de X, Y y Z en el espacio según sistema de referencia
107     for px in pos_pix2:
108         # Posición en X * escalado
109         x = (mtx_xi[px[0], px[1]] * scale_factor_x) + offset_x
110         # Posición en Y * escalado
111         y = (mtx_yi[px[0], px[1]] * scale_factor_y) + offset_y
112         # Posición en Z * escalado
113         z = (mtx_zi[px[0], px[1]] * scale_factor_z) + offset_z
114
115         # Guardamos en la lista la posición en X, Y y Z de cada pixel
116         perfil_puntos.append([x, y, z])
117
118     # Retornamos la lista de puntos
119     return np.array(perfil_puntos)
120
121 # Leemos imagen del directorio
122 img = cv2.imread("D:/Trabajo Final (Perfilometro)/Programa/ImgsPerfilometro/WIN_20240208_21_51_56_Pro.jpg")
123 #img = cv2.imread("D:/Trabajo Final (Perfilometro)/Programa/ImgsPerfilometro/WIN_20240218_16_16_26_Pro.jpg")
124
125 # Mostramos imagen original
```

```
126 cv2.imshow("Original", img)
127 cv2.waitKey(0)
128
129 # Procesamos imagen y obtenemos el perfil
130 puntos_perfil = PROCESAR(img)
131
132 # Graficamos los puntos obtenidos
133 plt.figure(figsize=(20, 10)) # Ajustamos el tamaño del gráfico
134 plt.plot(puntos_perfil[:, 2], puntos_perfil[:, 1], linestyle="dashed", color="red")
135 plt.xticks(range(-15, 35, 1)) # Ajustamos los intervalos de los números en los ejes x y y
136 plt.yticks(range(-5, 30, 1))
137 plt.grid()
138 plt.xlabel("Ancho [mm]")
139 plt.ylabel("Alto [mm]")
140 plt.xlim(-15, 35)
141 plt.ylim(-5, 30)
142 plt.show()
143
144
145
146
```

Calculo de altura del perfil

```
In [ ]: 1 # Calculamos la altura
2 puntos_encima_2mm = puntos_perfil[puntos_perfil[:, 1] > 2] # Filtramos los puntos por encima de 2 mm en el eje Y
3 puntos_debajo_2mm = puntos_perfil[puntos_perfil[:, 1] < 2] # Filtramos los puntos por debajo de 2 mm en el eje Y
4
5 # Ordenamos los puntos filtrados por su valor en el eje X
6 puntos_encima_2mm_sorted = np.sort(puntos_encima_2mm, axis=0)
7 puntos_debajo_2mm_sorted = np.sort(puntos_debajo_2mm, axis=0)
8
9 # Calculamos los índices que representan el 10% de los valores más bajos y altos para los puntos por encima de 2 m
10 n_encima = len(puntos_encima_2mm_sorted)
11 low_index_encima = int(n_encima * 0.1)
12 high_index_encima = int(n_encima * 0.9)
13
14 # Seleccionamos solo los valores dentro del rango del 10% al 90% para los puntos por encima de 2 mm
15 puntos_80_percent_encima = puntos_encima_2mm_sorted[low_index_encima:high_index_encima]
16
17 # Calculamos el promedio de los valores seleccionados para los puntos por encima de 2 mm
18 altura_promedio_80_percent_encima = np.mean(puntos_80_percent_encima[:, 1])
19
20 # Realizamos el mismo proceso para los puntos por debajo de 2 mm
21 n_debajo = len(puntos_debajo_2mm_sorted)
22 low_index_debajo = int(n_debajo * 0.1)
23 high_index_debajo = int(n_debajo * 0.9)
24 puntos_80_percent_debajo = puntos_debajo_2mm_sorted[low_index_debajo:high_index_debajo]
25 altura_promedio_80_percent_debajo = np.mean(puntos_80_percent_debajo[:, 1])
26
27 # Calculamos la diferencia entre los promedios de los puntos por encima y por debajo de 2 mm
28 diferencia_altura_promedio = round(altura_promedio_80_percent_encima - altura_promedio_80_percent_debajo, 1)
29
30 # Mostramos los resultados
31 print("Altura promedio del perfil (80% de los valores por encima de 2 mm):", round(altura_promedio_80_percent_enci
32 print("Altura promedio de la base (80% de los valores por debajo de 2 mm):", round(altura_promedio_80_percent_deba
33 print("Diferencia de alturas promedio:", diferencia_altura_promedio, "mm")
34
```


Calculo del ancho del perfil

```
In [ ]: 1 # Encontrar el inicio y el final del perfil
2 inicio_perfil = None
3 fin_perfil = None
4 for i in range(len(puntos_perfil)):
5     # Si el valor de Y es mayor a 2 mm, encontramos el inicio del perfil
6     if puntos_perfil[i, 1] > 2:
7         inicio_perfil = i
8         break
9
10 for i in range(len(puntos_perfil) - 1, -1, -1):
11     # Si el valor de Y es mayor a 2 mm, encontramos el final del perfil
12     if puntos_perfil[i, 1] > 2:
13         fin_perfil = i
14         break
15
16 # Calcular el ancho del perfil
17 if inicio_perfil is not None and fin_perfil is not None:
18     ancho_perfil = puntos_perfil[inicio_perfil, 2] - puntos_perfil[fin_perfil, 2]
19     print("Ancho del perfil:", ancho_perfil, "mm")
20 else:
21     print("No se encontraron puntos para calcular el ancho del perfil.")
```

Graficamos el perfil con los datos obtenidos

```
In [ ]: 1 # Graficar el perfil con datos:
2 plt.figure(figsize=(20, 10)) # Ajustamos el tamaño del gráfico
3 plt.plot(puntos_perfil[:, 2], puntos_perfil[:, 1], linestyle="dashed", color="red")
4
5 # Dibujar flecha para el ancho
6 plt.annotate('', xy=(puntos_perfil[fin_perfil, 2], altura_promedio_80_percent_encima + 2),
7                xytext=(puntos_perfil[inicio_perfil, 2], altura_promedio_80_percent_encima + 2),
8                arrowprops=dict(arrowstyle='<->', color='green'))
9 plt.text((puntos_perfil[inicio_perfil, 2] + puntos_perfil[fin_perfil, 2]) / 2,
10          altura_promedio_80_percent_encima + 2.5, 'Ancho del perfil Promedio: {} mm'.format(round(ancho_perfil, 1))
11          color='green', fontsize=12, ha='center')
12
13 # Dibujar flecha para la altura promedio
14 plt.annotate('', xy=(0, altura_promedio_80_percent_debajo),
15                xytext=(0, altura_promedio_80_percent_encima),
16                arrowprops=dict(arrowstyle='<->', color='blue'))
17 plt.text(0.5, (altura_promedio_80_percent_encima + altura_promedio_80_percent_debajo) / 2,
18          f'Diferencia de alturas promedio: {diferencia_altura_promedio} mm', color='blue', fontsize=12, ha='left')
19
20 plt.xticks(range(-15, 35, 1)) # Ajustamos los intervalos de los números en los ejes x y y
21 plt.yticks(range(-5, 30, 1))
22 plt.grid()
23 plt.xlabel("Ancho [mm]")
24 plt.ylabel("Alto [mm]")
25 plt.xlim(-15, 35)
26 plt.ylim(-5, 30)
27 plt.show()
```

INTERFACE ARDUINO

In []:

```
1 import serial
2 import time
3 import numpy as np
4 import cv2
5 import matplotlib.pyplot as plt
6
7 # Configurar puerto serie (Cambia 'COM4' si es necesario)
8 ser = serial.Serial('COM4', 115200, timeout=1) # Windows: 'COM4', Linux/Mac: '/dev/ttyUSB0'
9 time.sleep(2) # Esperar a que el puerto se estabilice
10
11 print("Esperando comandos de Arduino...")
12
13 #-----
14
15 # Importamos librerías
16
17 # Cargamos los archivos de las matrices de posición X, Y y Z
18 mtx_xi = np.loadtxt("mtx_x_izq.txt") # Matriz pos. pixeles en X
19 mtx_yi = np.loadtxt("mtx_y_izq.txt") # Matriz pos. pixeles en Y
20 mtx_zi = np.loadtxt("mtx_z_izq.txt") # Matriz pos. pixeles en Z
21
22 #-----
23
24 # Creamos las variables auxiliares a utilizar en la ejecución del programa
25 scale_factor_x = 1 # Escalado en X
26 scale_factor_y = 2.3 # Escalado en Y
27 scale_factor_z = 2.3 # Escalado en Z
28
29 #-----
30
31 # Creamos los offset en X, Y y Z para mover el grafico.
32 offset_x = 0
33 offset_y = -18
34 offset_z = 0
35
36 #-----
37
38 # Creamos la función para filtrado de "ROJO" mediante HSV
39 def HSV_FILTRO(image):
40     # Duplicamos la imagen
41     result = image.copy()
```

```

42     # Convertimos la imagen a espacio de color HSV
43     hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
44     # Definimos los límites inferiores y superiores del color rojo en HSV
45     lower_red = np.array([0, 50, 50])
46     upper_red = np.array([10, 255, 255])
47     # Creamos la máscara para el color rojo
48     mask1 = cv2.inRange(hsv_image, lower_red, upper_red)
49     # Definimos un rango adicional para el color rojo debido a su disposición en el espacio de color HSV
50     lower_red = np.array([170, 50, 50])
51     upper_red = np.array([180, 255, 255])
52     # Creamos otra máscara para el color rojo
53     mask2 = cv2.inRange(hsv_image, lower_red, upper_red)
54     # Combinamos ambas máscaras para obtener una única máscara para el rojo
55     mask = cv2.bitwise_or(mask1, mask2)
56     # Aplicamos la máscara a la imagen original para obtener solo los píxeles rojos
57     result = cv2.bitwise_and(result, result, mask=mask)
58     # Convertimos la imagen resultante a escala de grises
59     img_gris = cv2.cvtColor(result, cv2.COLOR_BGR2GRAY)
60     # Retornamos la imagen filtrada
61     return img_gris
62
63     #-----
64
65     # Creamos la función para calibración de imagen
66     def CALIBRAR(imagen, h, w):
67         # PROPIEDADES INTRÍNSECAS DE CÁMARA
68         # Matriz intrínseca de Cámara
69         M_int = np.array([[1.38106997e03, 0.00000000e00, 6.87317037e02],
70                          [0.00000000e00, 1.38209804e03, 3.82421364e02],
71                          [0.00000000e00, 0.00000000e00, 1.00000000e00]])
72
73         # Matriz de Distorsión
74         D = np.array([8.30863128e-02, 1.57413825e00, 4.52467136e-04, 5.90615097e-03, -7.07611874e00])
75
76         # Matriz Transformación de Cámara - Pixel
77         newcameramtx, roi = cv2.getOptimalNewCameraMatrix(M_int, D, (w, h), 1, (w, h), centerPrincipalPoint=False)
78
79         # Matriz Imagen calibrada
80         dst = cv2.undistort(imagen, M_int, D, None, newcameramtx)
81
82         # Se ajusta tamaño de imagen
83         x, y, w, h = roi

```

```

84     dst = dst[y : y + h, x : x + w]
85
86     # Retorna imagen calibrada
87     return dst
88
89     #-----
90
91     # Creamos la función para procesar la imagen
92     def PROCESAR(image):
93         # Inicializamos las variables a utilizar
94         perfil_puntos = [] # Lista de puntos del perfil de la pieza
95         pos_pix2 = [] # Lista de cada pixel a encontrar en las matrices de referencia
96
97         # Obtenemos el tamaño de la imagen recibida
98         h, w = image.shape[:2]
99
100        # Procesamos la imagen -> calibramos y filtramos "ROJO"
101        image = CALIBRAR(image, h, w)
102        image = HSV_FILTRO(image)
103
104        # Aplicamos un tresh para poner los pixeles en 1 o 0 según los valores límites fijados
105        image = cv2.threshold(image, 15, 255, cv2.THRESH_BINARY)[1]
106
107        # Recorremos todas las columnas de la imagen, para asegurarnos obtener una linea lo más fina posible a la curva
108        img_columnas = len(image[0]) # Obtenemos la cantidad de columnas que tiene la imagen
109
110        for col in range(0, img_columnas):
111            vpx = np.where(image[:, col] != 0) # Aquellos pixeles con "1" son los pixeles que queremos ubicar
112
113            # Si para dicha columna tenemos pixeles en "1", entonces lo guardamos en la lista
114            if len(vpx[0]) > 0:
115                # Ubicamos el pixel más centrado de la linea para ajustar lo más posible.
116                if len(vpx[0]) % 2 == 0:
117                    pos_pix2.append([vpx[0][int(len(vpx[0]) / 2)], col])
118                else:
119                    pos_pix2.append([vpx[0][int(round(len(vpx[0]) / 2, 0))], col])
120
121        # Para cada posición encontrada, se obtienen los valores de X, Y y Z en el espacio según sistema de referencia
122        for px in pos_pix2:
123            # Posición en X * escalado
124            x = (mtx_xi[px[0], px[1]] * scale_factor_x) + offset_x
125            # Posición en Y * escalado

```

```
126     y = (mtx_yi[px[0], px[1]] * scale_factor_y) + offset_y
127     # Posición en Z * escalado
128     z = (mtx_zi[px[0], px[1]] * scale_factor_z) + offset_z
129
130     # Guardamos en la lista la posición en X, Y y Z de cada pixel
131     perfil_puntos.append([x, y, z])
132
133     # Retornamos la lista de puntos
134     return np.array(perfil_puntos)
135
136     #-----
137
138     # Esperar hasta que Arduino envíe "MEDIR"
139     while True:
140         if ser.in_waiting > 0:
141             comando = ser.readline().decode('utf-8').strip()
142             print(f"Recibido comando: {comando}")
143
144             if comando == "MEDIR":
145                 try:
146
147                     # Capturar imagen desde la cámara
148                     cap = cv2.VideoCapture(0) # Usar cámara por defecto (ajustar índice si es necesario)
149                     cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
150                     cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
151
152                     if not cap.isOpened():
153                         print("Error: No se pudo abrir la cámara.")
154                         ser.write("0 0\r\n".encode('utf-8'))
155                         continue
156
157                     ret, frame = cap.read()
158                     cap.release()
159
160                     if not ret:
161                         print("Error: No se pudo capturar la imagen.")
162                         ser.write("0 0\r\n".encode('utf-8'))
163                         continue
164
165                     # Guardar imagen capturada como 'profile.jpg', sobrescribiendo si ya existe
166                     ruta_imagen = "D:/UNLZ/4 - Proyecto final (1C2025)/Phyton/Programa/ImgsPerfilometro/profile.jpg"
167                     cv2.imwrite(ruta_imagen, frame)
```

```
168
169     # Cargar La imagen recién guardada para procesarla
170     img = cv2.imread(ruta_imagen)
171
172
173     # Procesa imagen SOLO cuando Arduino lo solicite
174     #img = cv2.imread("D:/Trabajo Final (Perfilometro)/Programa/ImgsPerfilometro/WIN_20240208_21_51_56
175     #img = cv2.imread("D:/Trabajo Final (Perfilometro)/Programa/ImgsPerfilometro/WIN_20240218_16_16_26
176
177     if img is None:
178         print("Error: No se pudo cargar la imagen.")
179         ser.write("0 0\r\n".encode('utf-8'))
180         continue
181
182     # Procesamos imagen y obtenemos el perfil
183     puntos_perfil = PROCESAR(img)
184
185     #-----
186
187     # CALCULO ALTURA PERFIL
188     puntos_encima_2mm = puntos_perfil[puntos_perfil[:, 1] > 2] # Filtramos Los puntos por encima de 2
189     puntos_debajo_2mm = puntos_perfil[puntos_perfil[:, 1] < 2] # Filtramos Los puntos por debajo de 2
190
191     # Ordenamos Los puntos filtrados por su valor en el eje X
192     puntos_encima_2mm_sorted = np.sort(puntos_encima_2mm, axis=0)
193     puntos_debajo_2mm_sorted = np.sort(puntos_debajo_2mm, axis=0)
194
195     # Calculamos Los índices que representan el 10% de Los valores más bajos y altos para Los puntos p
196     n_encima = len(puntos_encima_2mm_sorted)
197     low_index_encima = int(n_encima * 0.1)
198     high_index_encima = int(n_encima * 0.9)
199
200     # Seleccionamos solo Los valores dentro del rango del 10% al 90% para Los puntos por encima de 2 m
201     puntos_80_percent_encima = puntos_encima_2mm_sorted[low_index_encima:high_index_encima]
202
203     # Calculamos el promedio de Los valores seleccionados para Los puntos por encima de 2 mm
204     altura_promedio_80_percent_encima = np.mean(puntos_80_percent_encima[:, 1])
205
206     # Realizamos el mismo proceso para Los puntos por debajo de 2 mm
207     n_debajo = len(puntos_debajo_2mm_sorted)
208     low_index_debajo = int(n_debajo * 0.1)
209     high_index_debajo = int(n_debajo * 0.9)
```



```

210 puntos_80_percent_debajo = puntos_debajo_2mm_sorted[low_index_debajo:high_index_debajo]
211 altura_promedio_80_percent_debajo = np.mean(puntos_80_percent_debajo[:, 1])
212
213 # Calculamos la diferencia entre los promedios de los puntos por encima y por debajo de 2 mm
214 diferencia_altura_promedio = round(altura_promedio_80_percent_encima - altura_promedio_80_percent_
215
216 #-----
217
218 # CALCULO ANCHO PERFIL
219 # Encontrar el inicio y el final del perfil
220 inicio_perfil = None
221 fin_perfil = None
222 for i in range(len(puntos_perfil)):
223     # Si el valor de Y es mayor a 2 mm, encontramos el inicio del perfil
224     if puntos_perfil[i, 1] > 2:
225         inicio_perfil = i
226         break
227
228 for i in range(len(puntos_perfil) - 1, -1, -1):
229     # Si el valor de Y es mayor a 2 mm, encontramos el final del perfil
230     if puntos_perfil[i, 1] > 2:
231         fin_perfil = i
232         break
233
234 # Calcular el ancho del perfil
235 if inicio_perfil is not None and fin_perfil is not None:
236     ancho_perfil = puntos_perfil[inicio_perfil, 2] - puntos_perfil[fin_perfil, 2]
237
238 #-----
239
240 # GRAFICO EL PERFIL
241
242 plt.figure(figsize=(20, 10)) # Ajustamos el tamaño del gráfico
243 plt.plot(puntos_perfil[:, 2], puntos_perfil[:, 1], linestyle="dashed", color="red")
244
245 # Dibujar flecha para el ancho
246 plt.annotate('', xy=(puntos_perfil[fin_perfil, 2], altura_promedio_80_percent_encima + 2),
247               xytext=(puntos_perfil[inicio_perfil, 2], altura_promedio_80_percent_encima + 2),
248               arrowprops=dict(arrowstyle='<->', color='green'))
249 plt.text((puntos_perfil[inicio_perfil, 2] + puntos_perfil[fin_perfil, 2]) / 2,
250         altura_promedio_80_percent_encima + 2.5, 'Ancho del perfil Promedio: {} mm'.format(round(
251         color='green', fontsize=12, ha='center')

```

```

252
253     # Dibujar flecha para la altura promedio
254     plt.annotate('', xy=(0, altura_promedio_80_percent_debajo),
255                  xytext=(0, altura_promedio_80_percent_encima),
256                  arrowprops=dict(arrowstyle='<->', color='blue'))
257     plt.text(0.5, (altura_promedio_80_percent_encima + altura_promedio_80_percent_debajo) / 2,
258             f'Diferencia de alturas promedio: {diferencia_altura_promedio} mm', color='blue', fontsize=12)
259
260     plt.xticks(range(-15, 35, 1)) # Ajustamos los intervalos de los números en los ejes x y y
261     plt.yticks(range(-5, 30, 1))
262     plt.grid()
263     plt.xlabel("Ancho [mm]")
264     plt.ylabel("Alto [mm]")
265     plt.xlim(-15, 35)
266     plt.ylim(-5, 30)
267     plt.show()
268
269     #-----
270
271     # ENVIO VALORES A ARDUNIO
272     mensaje = f"{ancho_perfil} {diferencia_altura_promedio}\r\n"
273     ser.write(mensaje.encode('utf-8'))
274     print(f"Enviando medición: Ancho={ancho_perfil}, Alto={diferencia_altura_promedio}")
275
276     except Exception as e:
277         print(f"Error al calcular perfil: {e}")
278         ser.write("0 0\r\n".encode('utf-8'))
279
280

```

In []:

1

In []:

1

